
Universidad “Mayor de San Andrés”
Facultad de Ciencias Puras y Naturales



Informe Técnico Actividad Individual Multimedia

Universitarios: Misme Flores Johan Ludvig

Carrera: Informática

Docente: Ph.D. Moises Martin Silva Choque

Paralelo: “A”

Fecha: 15 de junio del 2025

La Paz - Bolivia

1. Introducción

Este informe documenta el desarrollo de una aplicación web interactiva orientada al procesamiento de imágenes digitales, construida como tarea individual para la materia de Multimedia e Inteligencia Artificial. El proyecto integra tres módulos funcionales distintos dentro de una misma plataforma, accesible desde el navegador sin necesidad de instalar software adicional por parte del usuario.

El sistema fue desarrollado completamente en Python usando el framework Flask para el servidor web, y la librería OpenCV junto con NumPy para el procesamiento de imágenes. La interfaz es responsiva, con un diseño moderno y navegación clara entre los módulos.

2. Estructura del proyecto

El proyecto sigue el patrón estándar de Flask, separando el código de servidor, los archivos estáticos y las plantillas HTML:

Ruta / Archivo	Descripción
Tarea Individual J/	Raíz del proyecto
app.py	Servidor Flask — lógica principal y rutas
requirements.txt	Dependencias Python del proyecto
templates/	Plantillas HTML (Jinja2)
base.html	Plantilla base con navbar y estructura común
index.html	Página de inicio con tarjetas de navegación
clasificacion.html	Módulo A: Clasificación de texturas por clic
suavizado.html	Módulo B: Filtro de suavizado promedio 3x3
vaca_lola.html	Módulo C: Video multimedia La Vaca Lola
static/css/styles.css	Estilos globales (diseño oscuro, responsivo)
static/js/scripts.js	Scripts JavaScript: drag & drop, preview
static/uploads/	Imágenes cargadas y procesadas
static/videos/	Archivos de video (vaca.mp4)

3. Metodología Utilizada

El desarrollo siguió un enfoque modular: cada funcionalidad fue implementada de forma independiente y luego integrada en la aplicación. A continuación se explica la lógica y el código de cada módulo.

3.1. Módulo A — Clasificación de Texturas

Simula una clasificación satelital de superficies usando el espacio de color HSV. El usuario hace clic en cualquier punto de la imagen y el sistema identifica qué tipo de superficie es.

Paso 1 — Función de validación de archivos

Antes de procesar nada, se verifica que el archivo subido tenga una extensión permitida:

```
EXTENSIONES_PERMITIDAS = {'png', 'jpg', 'jpeg', 'bmp'}

def archivo_permitido(nombre_archivo):
    return '.' in nombre_archivo and \
        nombre_archivo.rsplit('.', 1)[1].lower() in
EXTENSIONES_PERMITIDAS
```

Lo que hace esta función es sencillo: separa el nombre del archivo por el último punto y comprueba si la extensión resultante está en la lista de permitidas. Por ejemplo, foto.JPG pasa a ser 'jpg' y sí está permitida; documento.pdf no está en la lista y es rechazado.

Paso 2 — Ruta de carga de la imagen

Cuando el usuario sube una imagen, Flask la guarda en disco con un nombre seguro y la muestra en pantalla:

```
@aplicacion.route('/clasificacion', methods=['GET', 'POST'])
def clasificacion():
    if request.method == 'POST':
        archivo = request.files['archivo']
        nombre_archivo = secure_filename(archivo.filename)
        ruta_entrada = os.path.join(
            aplicacion.config['CARPETA_SUBIDAS'],
            'in_clas_' + nombre_archivo
        )
        archivo.save(ruta_entrada)
        return render_template('clasificacion.html',
                               imagen_cargada='in_clas_' +
                               nombre_archivo)
```

`secure_filename()` limpia el nombre del archivo para evitar ataques de path traversal (por ejemplo, si alguien subiera un archivo llamado '.././config.py'). El prefijo 'in_clas_' diferencia las imágenes de clasificación de las de suavizado dentro de la carpeta de uploads.

Paso 3 — Clasificación del píxel (el núcleo del módulo)

Esta es la función más importante del módulo. Se activa cuando el usuario hace clic sobre la imagen:

```
@aplicacion.route('/clasificar_pixel', methods=['POST'])
def clasificar_pixel():
    datos = request.json
    x = int(datos.get('x', 0))
    y = int(datos.get('y', 0))
    nombre_imagen = datos.get('imagen', "")

    img = cv2.imread(ruta_imagen)

    # Extraer vecindad 5x5 alrededor del clic
    ventana = img[y_min:y_max, x_min:x_max]
    promedio_bgr = np.mean(ventana, axis=(0, 1))

    # Convertir a HSV para clasificación robusta
    pixel_promedio_bgr = np.uint8([[promedio_bgr]])
    pixel_hsv = cv2.cvtColor(pixel_promedio_bgr,
                             cv2.COLOR_BGR2HSV)[0][0]
    h, s, v = pixel_hsv[0], pixel_hsv[1], pixel_hsv[2]
```

En lugar de analizar un único píxel (que puede ser ruidoso), se toma una ventana de 5x5 píxeles alrededor del clic y se promedia. Luego se convierte de BGR (el formato interno de OpenCV) a HSV, donde H=tono de color, S=saturación y V=brillo. Esto es más útil porque separa el color del brillo, haciendo la clasificación menos sensible a sombras o iluminación desigual.

Paso 4 — Árbol de decisiones HSV

Con los valores H, S y V calculados, se aplica un árbol de decisiones para determinar la superficie:

```
# V < 50 → muy oscuro = sombra o asfalto
if v < 50:
    clase_detectada = 'Asfalto / Sombra'

# Poca saturación + muy brillante = blanco
elif s < 45 and v > 190:
    clase_detectada = 'Nubes / Nieve / Blanco'

# Poca saturación moderada = gris = cemento
```

```

elif s < 50 and v <= 190:
    clase_detectada = 'Cemento / Roca'

# Tiene color: evaluar el Tono (H)
elif h >= 38 and h < 85:
    clase_detectada = 'Césped / Vegetación'

elif h >= 85 and h < 110:
    if v > 160:
        clase_detectada = 'Cielo Despejado'
    else:
        clase_detectada = 'Agua (Lago / Río)'

# ...y así para tierra, arena, océano, etc.

```

La lógica sigue una jerarquía clara: primero filtra por brillo (V) para detectar zonas oscuras o blancas sin importar el color, luego filtra por saturación (S) para detectar grises, y finalmente usa el tono (H) para distinguir entre verdes, azules, rojizos, etc. Los valores H en OpenCV van de 0 a 179 (no de 0 a 360 como es habitual), por lo que los verdes están entre 38 y 85.

Clasificación de Texturas

Sube una imagen y haz clic en cualquier parte de ella para identificar la superficie (césped, tierra, cemento o asfalto) aplicando principios de distancia mínima de imágenes satelitales.

Imagen Cargada

Haz clic sobre un píxel de la imagen para clasificar su textura/color.



Resultado de la Detección

Superficie detectada: **Césped / Vegetación**

Color exacto extraído: 

Cargar otra imagen

3.2. Módulo B — Filtro de Suavizado Promedio 3×3

Este módulo aplica manualmente un filtro de promedio con ventana deslizante de 3×3 píxeles. La implementación es deliberadamente explícita (sin usar `cv2.blur`) para demostrar cómo funciona el algoritmo por dentro.

Paso 1 — Lectura y preparación de la imagen

Se carga la imagen y se crea una imagen de salida vacía del mismo tamaño:

```
def procesar_suavizado(ruta_imagen, ruta_salida):
    img = cv2.imread(ruta_imagen)
    if img is None:
        return False

    alto, ancho, canales = img.shape

    # Imagen de salida llena de ceros (negra)
    img_salida = np.zeros_like(img, dtype=np.float32)
```

Se usa `float32` (números decimales) en lugar de `uint8` (enteros 0-255) para la imagen de salida porque los promedios producen valores decimales. Al final se convierte de vuelta a `uint8`. Esto evita errores de redondeo o desbordamiento durante los cálculos.

Paso 2 — Borde de relleno (padding)

El problema con los píxeles del borde es que no tienen 8 vecinos completos. La solución es agregar un borde artificial de 1 píxel alrededor de la imagen:

```
# Añadir borde de 1 píxel usando reflexión
img_con_borde = cv2.copyMakeBorder(
    img,
    1, 1, 1, 1,      # top, bottom, left, right
    cv2.BORDER_REFLECT # refleja los píxeles del borde
)
img_con_borde = img_con_borde.astype(np.float32)
```

`BORDER_REFLECT` copia en espejo los píxeles del borde hacia afuera. Por ejemplo, si la fila superior tiene los colores [A, B, C], el borde añadido será [B, A] reflejado. Esto es mejor que rellenar con ceros (negro) porque evita un oscurecimiento artificial en los bordes de la imagen resultante.

Paso 3 — Bucle de ventana deslizante (el corazón del filtro)

Se recorre cada píxel de la imagen y se promedia su vecindad de 3×3 :

```
for i in range(1, alto + 1):
    for j in range(1, ancho + 1):

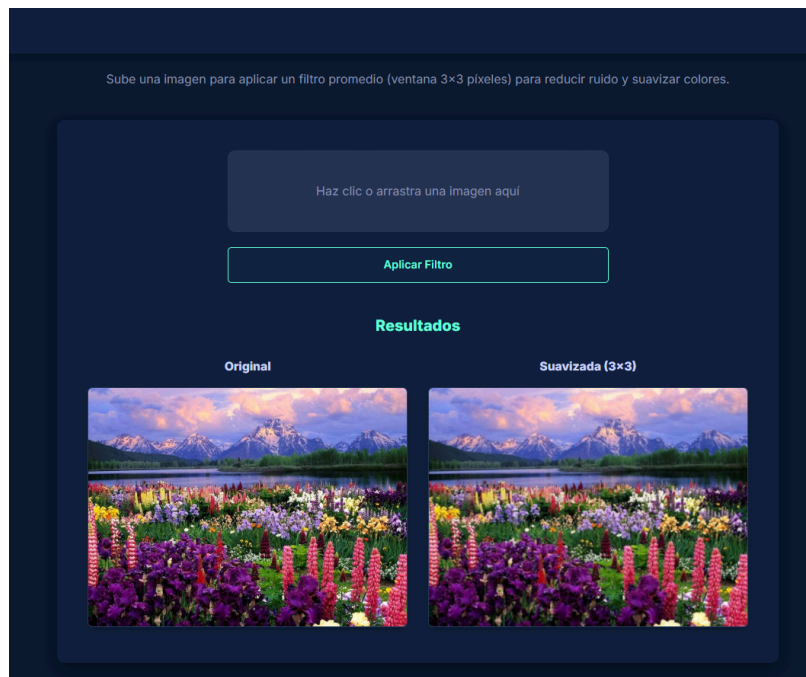
        # Extraer la ventana 3x3 centrada en (i, j)
        ventana = img_con_borde[i-1:i+2, j-1:j+2]

        # Calcular promedio de los 9 píxeles (en los 3 canales)
        img_salida[i-1, j-1] = np.mean(ventana, axis=(0, 1))

        # Convertir de vuelta a enteros 0-255
        img_salida = np.uint8(img_salida)
        cv2.imwrite(ruta_salida, img_salida)
```

El índice empieza en 1 (no en 0) porque `img_con_borde` tiene el borde extra de 1 píxel. Para cada posición (i, j) , el slicing `[i-1:i+2, j-1:j+2]` extrae exactamente los 9 píxeles de la vecindad 3×3 . La función `np.mean` con `axis=(0,1)` promedia en las dimensiones de fila y columna, devolviendo un vector de 3 valores (uno por canal B, G, R), que se asigna directamente al píxel de salida.

En términos matemáticos, lo que hace este bucle es una convolución con un kernel uniforme de 3×3 (todos los pesos iguales a $1/9$), también conocido como filtro de media o box filter.



3.3. Módulo C — Multimedia (La Vaca Lola)

Este módulo es una presentación multimedia simple. Toda su lógica está en el servidor Flask (una ruta sencilla) y en la plantilla HTML que reproduce el video:

```
# En app.py — ruta simple, sin procesamiento
@aplicacion.route('/vaca_lola')
def vaca_lola():
    return render_template('vaca_lola.html')

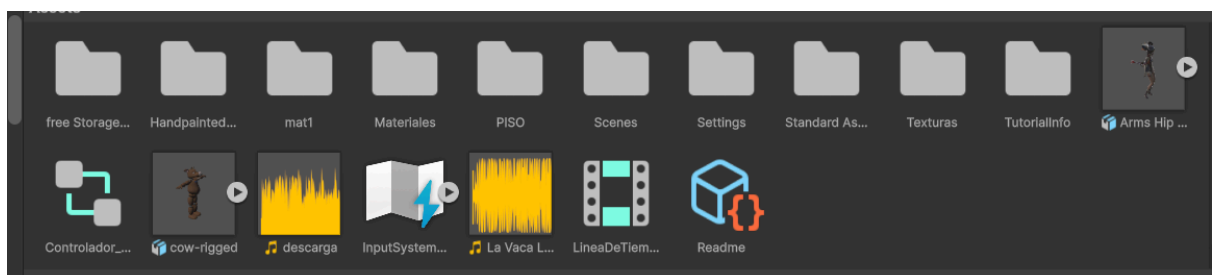
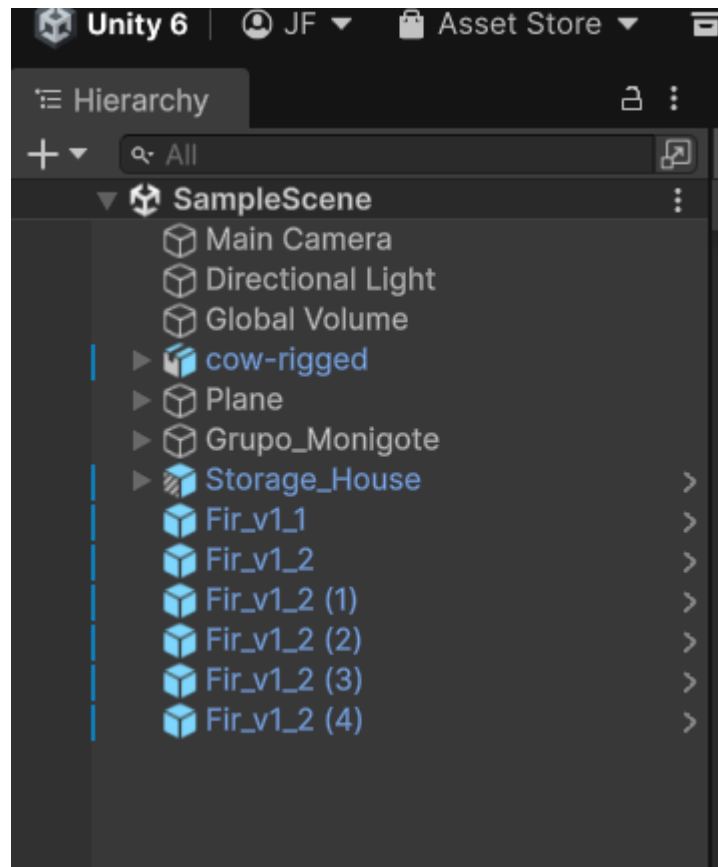
# En vaca_lola.html — reproductor HTML5
<video controls width='100%'>
  <source src="{{ url_for('static',
    filename='videos/vaca.mp4') }}"
    type='video/mp4'>
</video>
```

Este módulo describe el proceso de ensamble, rigging, animación y sincronización de audio para una producción multimedia 3D interactiva utilizando el motor Unity 6. La implementación se realiza de forma manual y explícita mediante componentes integrados como **Timeline** y el sistema de animación **Humanoid**, demostrando la lógica interna de la sincronización de clips.

Paso 1 — Preparación de la Escena y Asignación de Rigging (Humanoid)

Se cargan los assets 3D en la jerarquía de la escena. El problema principal con los esqueletos personalizados (cow-rigged y Arms Hip Hop Dance) es que Unity necesita estandarizar sus articulaciones para que compartan la misma animación:

- **Configuración del Rig:** Se seleccionan los archivos .fbx base en el panel Project, se accede a la pestaña **Rig** en el Inspector y se cambia el Animation Type a **Humanoid**, aplicando los cambios. Esto genera un **Avatar** para cada modelo, permitiendo que un esqueleto animal copia texturas de movimiento de un esqueleto humano (Mixamo) sin deformarse.
- **Componente Animator:** Cada personaje en la Hierarchy debe poseer un componente **Animator** apuntando a un **Animator Controller** común (en este caso, creado manualmente como Controlador_Baile). Aunque el controlador esté inicialmente vacío de estados lógicos complejos, su existencia es requerida para que el motor ceda el control de las articulaciones a la línea de tiempo.



Paso 2 — Construcción del Timeline y Mezcla de Audio

Para sincronizar el remix de "La Vaca Lola" con el baile, se crea un Asset de **Timeline** (LineaDeTiempo_Vaca) vinculado al director de la escena. La ventana de Timeline actúa como un secuenciador multipista no lineal:

- **Pista de Audio (Audio Track):** Se añade una pista de audio vacía a la secuencia y se arrastra el archivo de sonido del cover musical generado con **huggingface** y **Colab**. Esta pista se encarga de dictar el reloj maestro de la reproducción multimedia.

Código en Colab

```
from transformers import AutoProcessor,
MusicgenForConditionalGeneration
```

```
import scipy.io.wavfile

from IPython.display import Audio, display

import torch

import numpy as np


# 1. VARIABLE DEL MODELO DESTACADA

MODELO_HUGGINGFACE = "facebook/musicgen-small"

print(f"1. Cargando el modelo de Hugging Face: {MODELO_HUGGINGFACE}...")

device = "cuda" if torch.cuda.is_available() else "cpu"

processor =
AutoProcessor.from_pretrained(MODELO_HUGGINGFACE)

model =
MusicgenForConditionalGeneration.from_pretrained(MODELO_HUGGINGFACE).to(device)


# 2. CONFIGURACIÓN DEL PROMPT

prompt = "Cumbia version of the children's song La Vaca Lola, upbeat tropical cumbia rhythm with accordion, guacharaca, and infectious danceable bassline"

print(f"\n2. Generando la base musical para: '{prompt}'...")

inputs = processor(

    text=[prompt],

    padding=True,

    return_tensors="pt",
```

```

).to(device)

# Generamos unos ~10 segundos de audio (512 tokens)
audio_values = model.generate(**inputs,
max_new_tokens=512)

# Extraemos el audio y la frecuencia de muestreo
sampling_rate =
model.config.audio_encoder.sampling_rate

audio_base = audio_values[0, 0].cpu().numpy()

# 3. CREACIÓN DEL BUCLE (LOOP A 2 MINUTOS)

print("\n3. Multiplicando la pista para alcanzar los 2
minutos...")

duracion_base = len(audio_base) / sampling_rate
minutos_deseados = 2
segundos_deseados = minutos_deseados * 60

# Calculamos cuántas veces repetir el audio base
repeticiones = int(segundos_deseados / duracion_base)
+ 1

# Multiplicamos el array de audio
audio_bucle = np.tile(audio_base, repeticiones)

# Cortamos exactamente a los 2 minutos (120 segundos *
sampling_rate)

muestras_totales = segundos_deseados * sampling_rate
audio_final = audio_bucle[:muestras_totales]

```

```
# 4. GUARDAR Y REPRODUCIR

nombre_archivo = "vaca_lola_cumbia_2_min.wav"

scipy.io.wavfile.write(nombre_archivo,
rate=sampling_rate, data=audio_final)

print(f"\n;Proceso terminado! Archivo guardado como
'{nombre_archivo}'.")

print("Escucha tu cumbia de 2 minutos aquí:")

# Muestra el reproductor

display(Audio(audio_final, rate=sampling_rate))
```

```
[4] ✓ 31 s ▶ print("\n3. Multiplicando la pista para alcanzar los 2 minutos...")

duracion_base = len(audio_base) / sampling_rate
minutos_deseados = 2
segundos_deseados = minutos_deseados * 60

# Calculamos cuántas veces repetir el audio base
repeticiones = int(segundos_deseados / duracion_base) + 1

# Multiplicamos el array de audio
audio_bucle = np.tile(audio_base, repeticiones)

# Cortamos exactamente a los 2 minutos (120 segundos * sampling_rate)
muestras_totales = segundos_deseados * sampling_rate
audio_final = audio_bucle[:muestras_totales]

# 4. GUARDAR Y REPRODUCIR
nombre_archivo = "vaca_lola_cumbia_2_min.wav"
scipy.io.wavfile.write(nombre_archivo, rate=sampling_rate, data=audio_final)

print(f"\n¡Proceso terminado! Archivo guardado como '{nombre_archivo}'.")
print("Escucha tu cumbia de 2 minutos aquí:")

# Muestra el reproductor
display(Audio(audio_final, rate=sampling_rate))
```

1. Cargando el modelo de Hugging Face: facebook/musicgen-small...
Loading weights: 100% 611/611 [00:00<00:00]

2. Generando la base musical para: 'Cumbia version of the children's song La

3. Multiplicando la pista para alcanzar los 2 minutos...

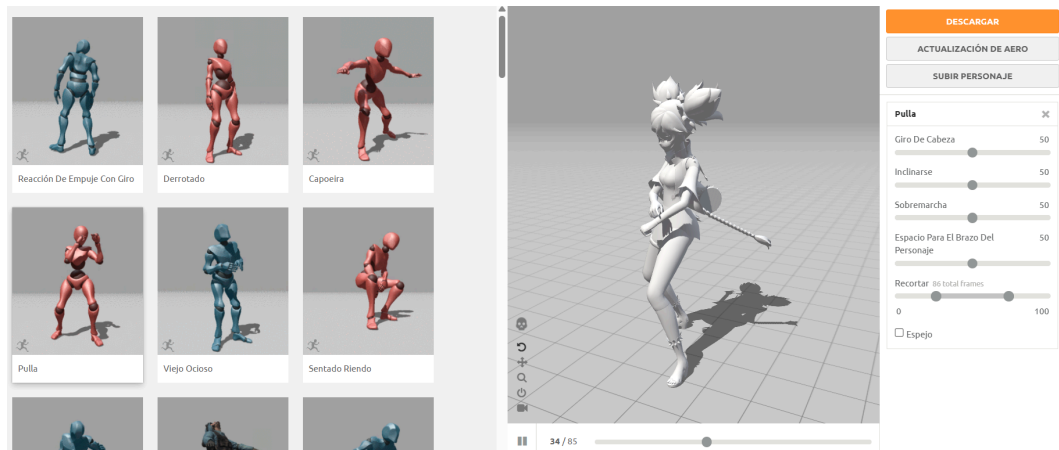
¡Proceso terminado! Archivo guardado como 'vaca_lola_cumbia_2_min.wav'.
Escucha tu cumbia de 2 minutos aquí:

0:00 / 2:00

- **Pistas de Animación (Animation Tracks):** Se crean dos pistas de animación independientes dentro del mismo Timeline. Se vincula el componente Animator de la vaca (cow-rigged) a la primera, y el del monigote gris (Arms Hip Hop Dance (1)) a la segunda.

Paso 3 — Inyección de Animación, Bucles y Alineación de Coordenadas (El Corazón del Módulo)

Se extrae el clip de animación puro (mixamo.com) contenido dentro del archivo .fbx del baile de Hip Hop y se inyecta en las dos pistas de animación del Timeline.



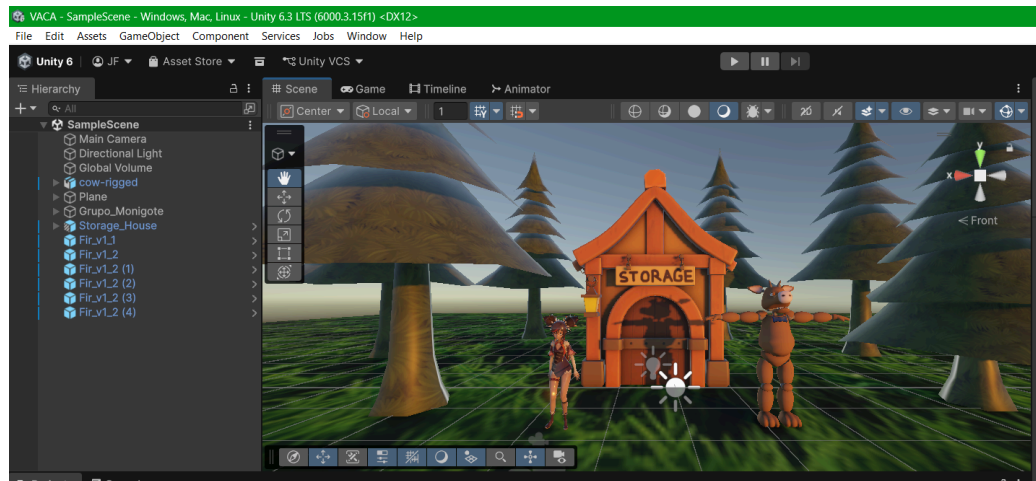
- **Sincronización Definitiva en el Eje Temporal:** Para lograr un baile al unísono, ambos bloques de animación se alinean de forma explícita en el fotograma cero (s: 0, f: 0), paralelos a la onda de sonido del audio.
- **Cálculo de Extrapolación (Loop):** Como la pista de audio es considerablemente más larga que el clip de Mixamo, se activa la propiedad **Loop Time** en el archivo de origen y se configura el **Post-Extrapolation** del Timeline en modo **Loop**. Esto permite estirar los bloques horizontalmente de forma manual hasta el segundo exacto 126.72 (final de la canción), repitiendo la coreografía cíclicamente sin microcortes.
- **Bloqueo de Desplazamiento (Apply Scene Offsets):** Por defecto, las animaciones de Mixamo fuerzan a los modelos a trasladarse al origen (0,0,0), provocando que los personajes se fusionen o se desalinen al presionar Play. Para solucionarlo, se accede a las propiedades de cada pista de animación en el Inspector y se cambia el parámetro **Track Offsets** a **Apply Scene Offsets**. Esto rompe el candado de la animación y obliga a Unity a respetar la posición manual asignada en la recta de la escena.

Paso 4 — Diseño de Entorno, Iluminación Cálida y Renderizado

Con la lógica de movimiento resuelta, se pule el apartado estético del videoclip:

- **Entorno 3D (Crops):** Se importa el paquete Cartoon Farm Crops desde el Package Manager (sección My Assets) y se distribuyen prefabs de maíz, zanahorias y calabazas en el fondo de la escena para contextualizar la temática campestre.
- **Iluminación del Sol:** Se selecciona la **Directional Light** de la escena. En su componente Light, se altera el espectro cromático de blanco a un tono **amarillo/naranja cálido** y se inclina su rotación en el eje **X** a unos 50°, simulando un atardecer que baña uniformemente todo el escenario.

- **Generación del Entregable (Unity Recorder):** Se instala la extensión oficial Unity Recorder desde el Unity Registry. Se añade un perfil de captura tipo **Movie**, configurando la salida en formato **MP4 (1920x1080)** con la casilla Include Audio activada. Al presionar Start Recording, el sistema reproduce la escena en tiempo real, compila el búfer de video y audio combinados, y exporta el archivo final reproducible para la entrega.



4. Herramientas y Tecnologías Empleadas

Componente	Detalle

Python 3.x	Lenguaje principal del servidor y del procesamiento analítico de imágenes.
Flask 3.0.0	Micro-framework web: enrutamiento, templates y servidor HTTP de la aplicación.
Werkzeug 3.0.1	Manejo seguro de archivos y strings cargados en el backend (<code>secure_filename</code>).
OpenCV 4.8.1	Lectura/escritura de imágenes, conversión matricial de espacios BGR → HSV.
NumPy 1.26.2	Operaciones matriciales para el filtro de suavizado y los promedios de ventana deslizante.
scikit-learn 1.3.2	Incluido como dependencia (disponible para extensiones de análisis futuras).
Jinja2	Motor de templates integrado en Flask para el renderizado dinámico de la interfaz HTML.
HTML5 / CSS3 / JS (ES6)	Interfaz web responsiva con diseño oscuro moderno y consumo asíncrono vía Fetch API.
Google Fonts (Inter)	Tipografía limpia y profesional adaptada para la interfaz de usuario.
Unity 6 (6000.3.15f1 LTS)	Motor de desarrollo 3D y entorno integrado de compilación multimedia interactiva.

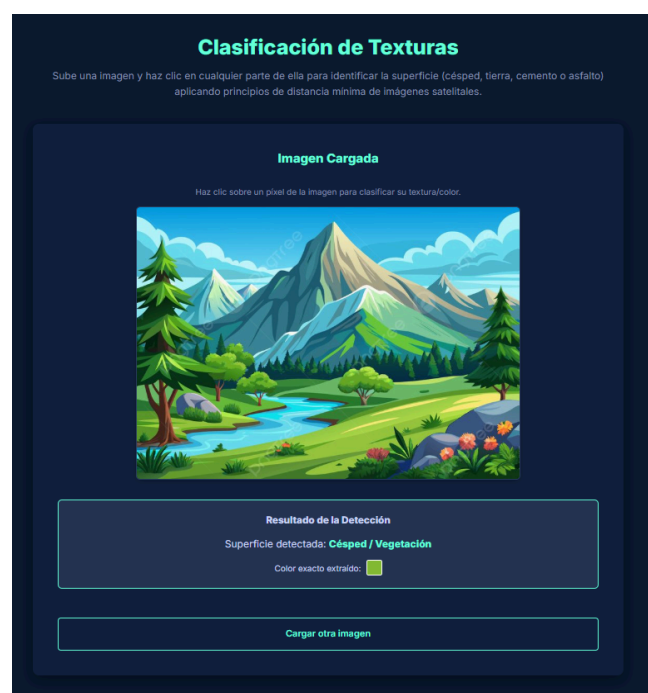
Unity Timeline	Secuenciador multipista no lineal encargado de la sincronización temporal de audio y animaciones.
Mecanim System (Humanoid)	Sistema de retargeting de Unity que transfiere la coreografía humana a proporciones animales.
Unity Recorder 1.0	Extensión oficial instalada vía <i>Unity Registry</i> para renderizar el búfer a video físico.
Mixamo FBX Core	Fuente de la animación base (<i>Arms Hip Hop Dance</i>) con curvas de animación pre-calculadas.
Universal Render Pipeline (URP)	Pipeline gráfico de Unity encargado de calcular materiales, sombras y la iluminación global.
Directional Light Component	Sistema de luz infinita configurado con espectro cromático cálido para simular el atardecer.
FFmpeg Integration	Backend nativo encargado de codificar y compilar el contenedor final de video en MP4 (H.264 + AAC).
Cartoon Farm Crops Pack	Paquete de assets 3D (<i>False Wisp Studios</i>) importado desde la <i>Asset Store</i> para el entorno.
Hugging Face (transformers)	Plataforma y librería principal utilizada para acceder, descargar y ejecutar modelos de inteligencia artificial de código abierto.

MusicGen (small)	Modelo de aprendizaje profundo encargado de la generación del audio y la base rítmica a partir de descripciones de texto.
Google Colab (T4 GPU)	Entorno de ejecución en la nube utilizado para procesar el modelo de inteligencia artificial utilizando aceleración por hardware.

5. Resultados Obtenidos

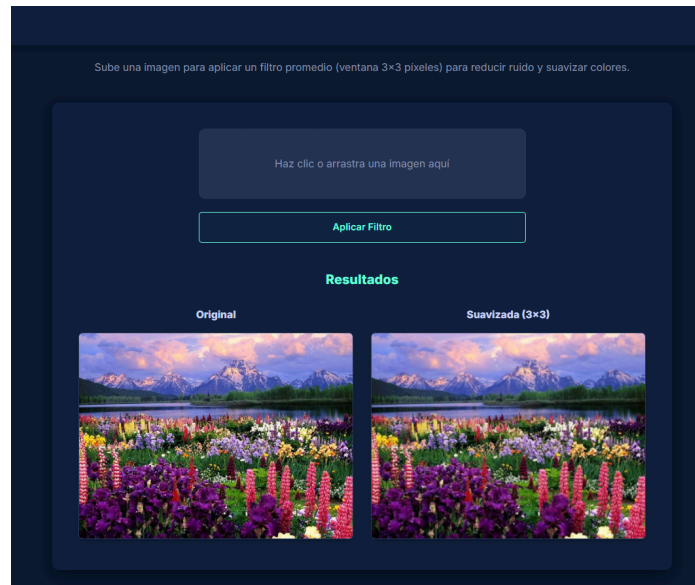
5.1. Módulo A — Clasificación de Texturas

El módulo funciona correctamente para imágenes con buena saturación de color. Las categorías más identificables son la vegetación (verdes, H: 38–85) y el cielo (azules claros, H: 85–110 con V alto). La clasificación es instantánea gracias a la comunicación asíncrona con el servidor vía fetch API.



5.2. Módulo B — Filtro de Suavizado

El filtro promedio de 3×3 produce una reducción visible del ruido de alta frecuencia y suaviza las transiciones entre colores. Al estar implementado con bucles Python puros, el tiempo de procesamiento es mayor para imágenes grandes, pero el resultado matemático es idéntico al de un box blur estándar.



5.3. Módulo C — Multimedia

El video se reproduce de forma fluida en el navegador usando el elemento HTML5. La página incluye la letra de la canción junto al reproductor.

